Shortest path

Dijkstra's algorithm

Dijkstra's algorithm

- **Dijkstra's algorithm** จะหาเส้นทางที่สั้นที่สุดจากโหนดเริ่มต้นไปยังทุกโหนดในกราฟเช่นเดียวกับ **Bellman–Ford algorithm**

- **ข้อดี** ของ Dijkstra's algorithm คืออัลกอริทึม**มีประสิทธิภาพมากกว่า**และสามารถ**ใช้กับกราฟขนาดใหญ่**ได้

- อย่างไรก็ตาม algorithm นี้จะทำงานได้ต้องไม่มี negative weight ในกราฟ

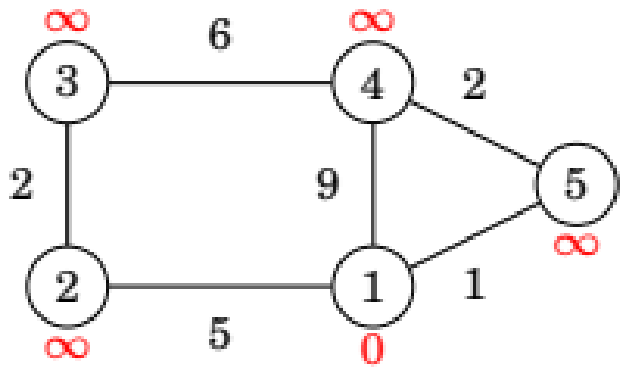
Dijkstra's algorithm

- เช่นเดียวกับ Bellman–Ford algorithm, **Dijkstra's algorithm** นั้นเก็บระยะทางที่ไปยังโหนดและลดค่ามันระหว่างการค้นหา

- **Dijkstra's algorithm** มีประสิทธิภาพเพราะว่ามันประมวลผลเส้นเชื่อมแต่ละเส้นในกราฟเพียงครั้งเดียว โดยใช้ความจริงที่ว่ากราฟไม่มี negative edges

ตัวอย่างการทำงาน

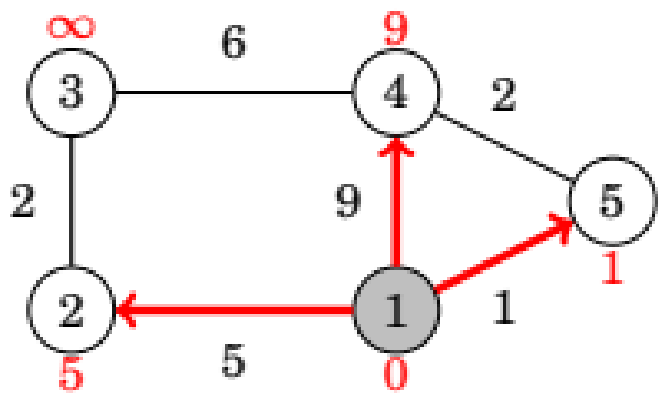
- พิจารณากราฟต่อไปนี้เราจะใช้ Dijkstra's algorithm เมื่อเริ่มต้นที่โหนด 1

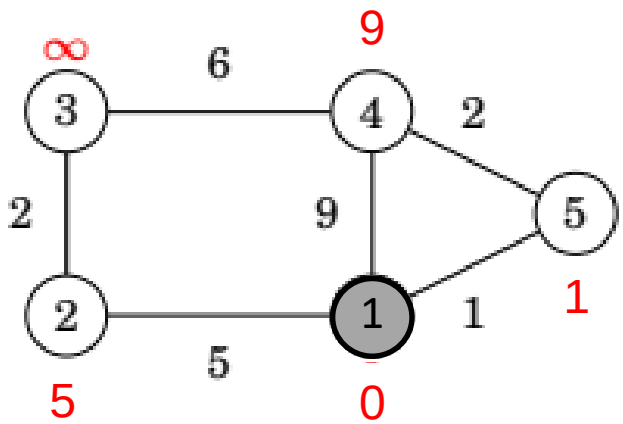


- เช่นเดียวกับ Bellman–Ford algorithm เริ่มต้นจะกำหนดระยะทางจากโหนดเริ่มต้นเป็น 0 และ โหนดที่เหลือเป็น infinity
- ในแต่ละรอบ Dijkstra's algorithm **จะเลือกโหนดที่ยังไม่ถูกพิจารณา** และมี**ระยะทางใกล้ที่สุด**
- ในตัวอย่างโหนดแรกเป็นโหนด 1 ซึ่งมีระยะทางเป็น 0

ตัวอย่างการทำงาน

- เมื่อโหนดถูกเลือก algorithm จะพิจารณาทุกเส้นเชื่อมที่เริ่มต้นที่โหนดนั้นและลดระยะทาง ตัวอย่างโหนดแรกเป็นโหนด 1 ซึ่งมีระยะทางเป็น 0

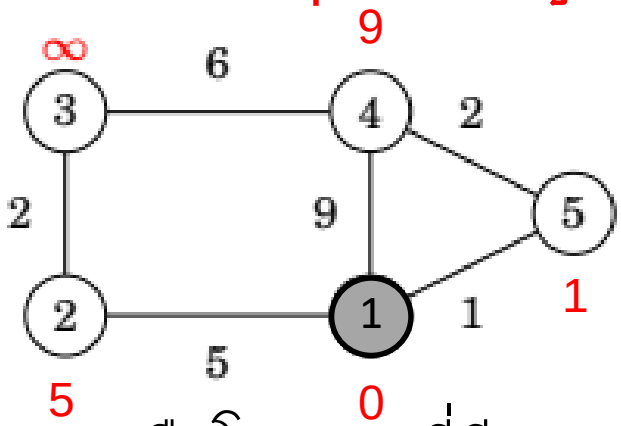




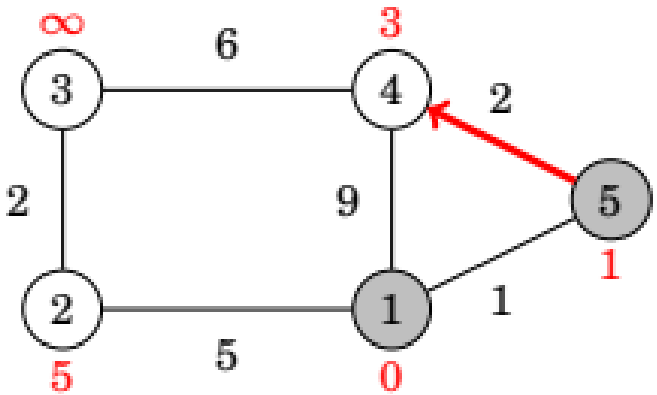
- ในกรณีนี้เส้นเชื่อมจากโหนด 1 ลดระยะทางของโหนด 2, 4 และ 5 ทำให้ระยะทางปัจจุบันเป็น 5, 9 และ 1

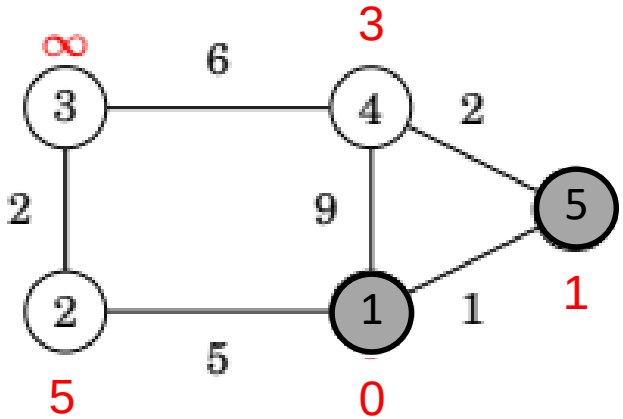
ตัวอย่างการทำงาน

- พิจารณาโหนดที่**ระยะทางน้อยที่สุด**ที่**ยังไม่ถูกแวะ**เป็นลำดับต่อไป



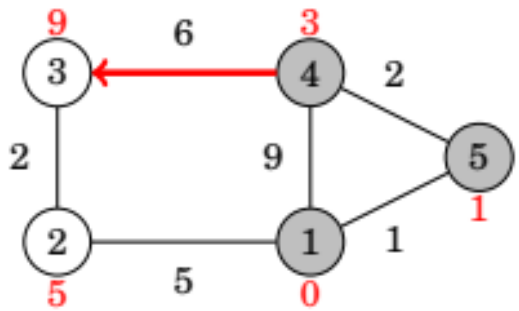
- โหนดต่อไปที่จะประมวลผลคือโหนด 5 ที่มีระยะทางเป็น 1 ซึ่งจะไปลดระยะทางที่ไปยังโหนด 4 จาก 9 –> 3

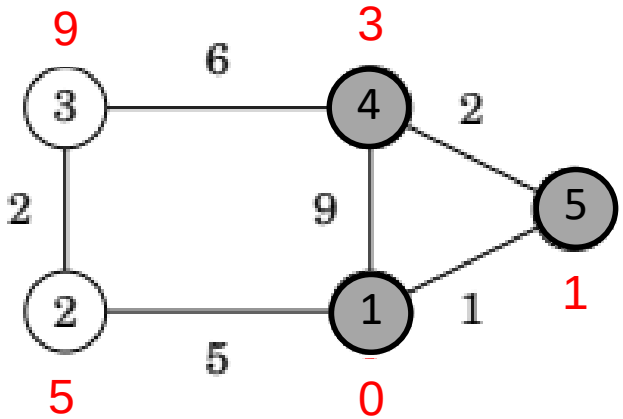




ตัวอย่างการทำงาน

- หลังจากนั้นโหนดต่อไปคือโหนด 4 จะลดระยะทางไปยังโหนด 3 ทำให้ระยะทางเป็น 9

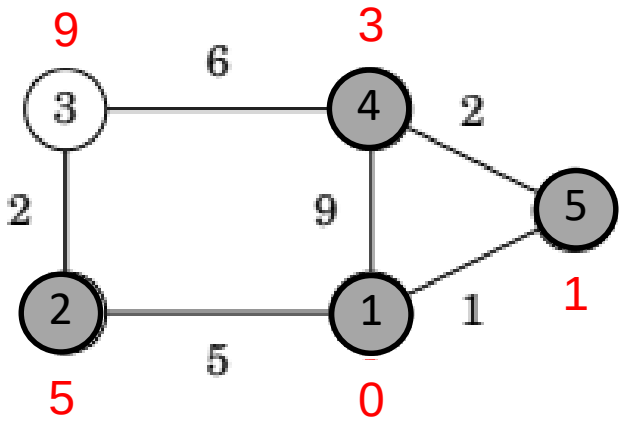


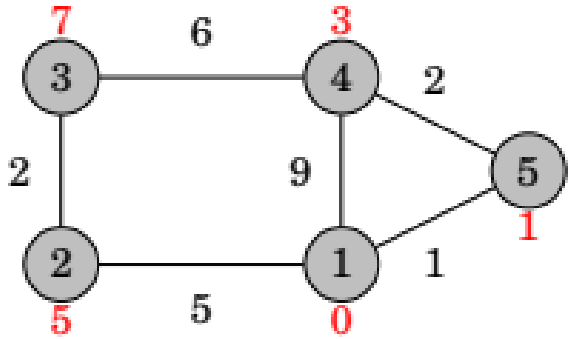


- คุณสมบัติของ Dijkstra's algorithm คือ**เมื่อโหนดถูกเลือกแล้วระยะทางของมันจะถือว่าสิ้นสุดแล้ว**
- ตัวอย่างเช่นที่จุดนี้ ระยะทาง 0 1 และ 3 เป็นระยะทางสุดท้ายของโหนด 1, 5 และ 4

ตัวอย่างการทำงาน

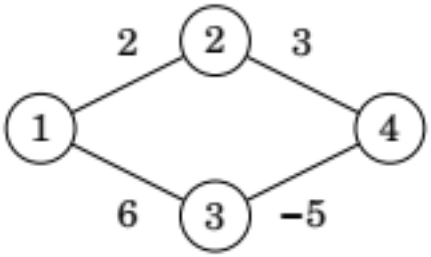
- หลังจากนั้น algorithm จะทำงานกับ 2 โหนดที่เหลือ ซึ่งได้ระยะทางสุดท้ายดังรูป





Negative edges

- ประสิทธิภาพของ Dijkstra's algorithm นั้นอยู่บนพื้นฐานของความจริงที่ว่าในกราฟ**ไม่มีเส้นเชื่อมที่มีน้ำหนักเป็นลบ** (negative edges)
- ถ้ามีเส้นเชื่อมเป็นลบ algorithm อาจจะให้ผลลัพธ์ที่ผิดได้ ตัวอย่างเช่นกราฟต่อไปนี้



- ระยะทางสั้นที่สุดจาก 1 ไปโหนด 4 คือ 1->3->4 ซึ่งความยาวเป็น 1 อย่างไรก็ตาม Dijkstra's algorithm หาเส้นทาง 1->2->4 ตามที่ค่าน้ำหนักน้อยที่สุดของเส้นเชื่อม

Implementation

- ต่อไปเป็น implementation ของ **Dijkstra's algorithm** ซึ่งคำนวณระยะทางสั้นที่สุดจากโหนด x ไปยังโหนดที่เหลือในกราฟ

- ใน**แต่ละรอบ**เราจะเอาข้อมูลของโหนดที่**ยังไม่ได้สำรวจที่ใกล้ x มากที่สุด**มาใช้ เราควรเก็บโหนดเหล่านี้ในโครงสร้างข้อมูลอะไร

- Dijkstra's algorithm จะเก็บข้อมูลของโหนดโดยใช้ priority queue สมมติว่าชื่อ pq เก็บ **pair** (เราควรให้<u>ข้อมูลตัวแรก</u>เป็นอะไร เพราะอะไร)
  - ข้อมูลตัวแรกเป็นระยะทางจากโหนดไปยัง source
  - ข้อมูลตัวที่สองเป็นหมายเลขโหนด

Implementation

- การเก็บโดยใช้ priority queue เก็บ pair แบบนี้ ทำให้เรา sort pq ตามระยะห่างจาก source จากน้อยไปมากได้ซึ่งถ้าเท่ากัน เราก็จะใช้หมายเลขโหนดแทน

- ทั้งนี้สามารถ implement ด้วย data structure ตัวอื่นได้ด้วยเช่นใช้ binary heap แต่ว่าไม่มีใน built-in library

- pq เริ่มต้นจะเก็บ 1 item นั่นคือ base case (0,s) ซึ่งเป็นจริงเพราะว่าเป็น source vertex ระยะทางก็เป็น 0 นั่นเอง

- จากนั้น Dijkstra's algorithm จะทำซ้ำจน pq หมด
  - หยิบข้อมูลจาก pq pair(d,u) จากตัวหน้าสุดของ pq เป็นการเลือกแบบ greedy
  - ถ้าระยะทาง d ของ u จาก source ที่ถูกบันทึกไว้ในมากกว่าระยะทาง dist[u] ก็จะไม่ทำอะไรกับ u แต่ถ้าไม่ใช่ เราก็จะปรับปรุงค่าของ u

- เมื่ออัลกอริทึมนี้ประมวลผลโหนด u มันจะพยายามปรับระยะทางของทุกๆ เพื่อนบ้าน v ของ u ให้ลดลง โดยการ relax เป็นการเซตค่า

$$dist[v] = min(dist[v],dist[u]+w(u,v))$$

- ทุกครั้งที่มีการ relax เส้นเชื่อม u->v
- เราจะต้อง
  - **push** pair(ระยะทางอันใหม่ที่สั้นกว่าของ v จาก source, v) เข้าไปใน pq
  - และ **pop** pair(ระยะทางอันเก่าที่ยาวกว่าของ v จาก source, v) ออกจาก pq

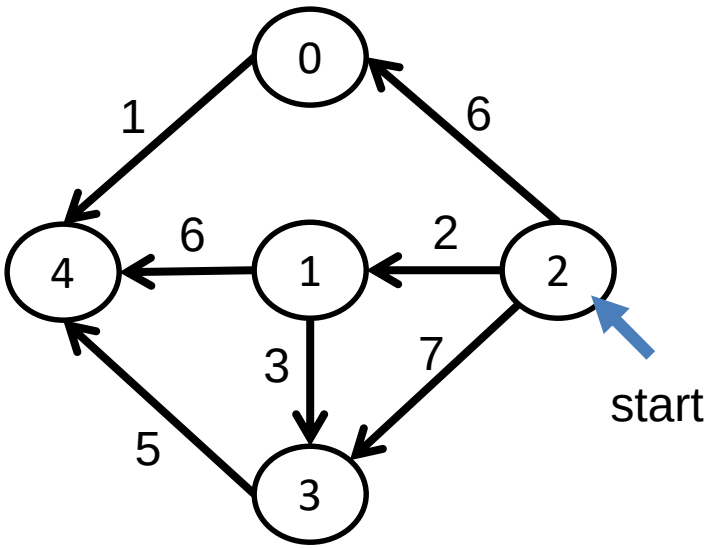
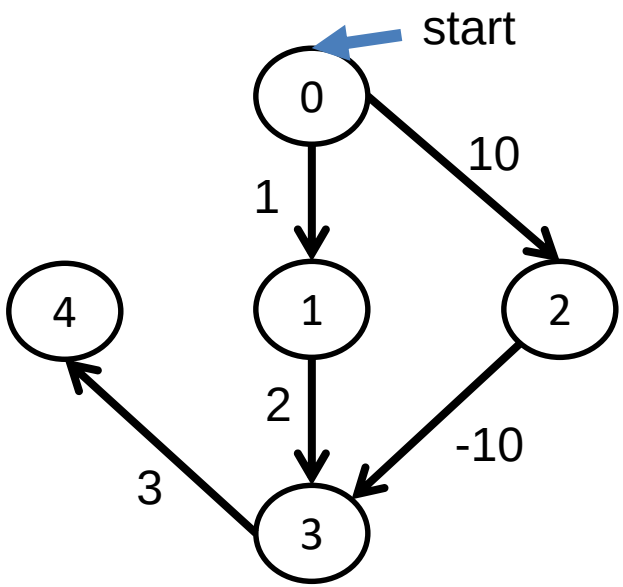
- ถ้าไม่ทำแบบก่อนหน้า ก็สามารถทำวิธีการนี้เรียกว่า **lazy deletion**
- ถ้าใคร relax แล้วดีขึ้นก็เพิ่มเข้า pq เลย คือการยอมให้มีมากกว่า 1 copy ของโหนดเดียวกันใน pq ที่มีระยะต่างกันจาก source
- ดังนั้นเราต้องเชคก่อนเพื่อที่จะประมวลผล pair ของข้อมูลโหนดที่ถูก dequeue ในครั้งแรก ซึ่งจะทำให้ได้ระยะทางที่ถูกและสั้น (ระยะทางของ copy อื่นนั้นจะไม่ทันสมัยและยาวกว่า
- code ต่อไปนี้ก็จะคล้าย **BFS** มากอยู่

```
#include <bits/stdc++.h>
using namespace std;
typedef pair<int, int> ii;
typedef vector<int> vi;
typedef vector<ii> vii;
#define INF 1000000000

int main() {
  int V, E, s, u, v, w;
  vector<vii> AdjList;
  scanf("%d %d %d", &V, &E, &s);

  AdjList.assign(V, vii());
  for (int i = 0; i < E; i++) {
    scanf("%d %d %d", &u, &v, &w);
    AdjList[u].push_back(ii(v, w));
// directed graph
 }
```

```
  vi dist(V, INF);
  dist[s] = 0;                        // INF = 1B
  priority_queue< ii, vector<ii>, greater<ii> > pq;
  pq.push(ii(0, s)); // เพื่อเก็บ pair มีระยะทางเพิ่มขึ้น
  while (!pq.empty()) {             // main loop
    ii front = pq.top();
    pq.pop();      // เอาตัวที่ระยะทางสั้นที่สุดออก
    int d = front.first, u = front.second;
    if (d > dist[u]) continue; //  เราใช้ lazy deletion ถ้าระยะมากกว่าที่เคยไปก็ไม่ทำ
    for (int j = 0; j < (int)AdjList[u].size(); j++) {
        ii v = AdjList[u][j]; // ดูเพื่อนบ้านทุกคนของ u
        if (dist[u] + v.second < dist[v.first]) {
          dist[v.first] = dist[u] + v.second; // relax operation
          pq.push(ii(dist[v.first], v.first));
        }
    }
  }
  for (int i = 0; i < V; i++)
    printf("SSSP(%d, %d) = %d\n", s, i, dist[i]);
  return 0;
}
```

https://bit.ly/2HE4RVI

- ทดลองกราฟนี้